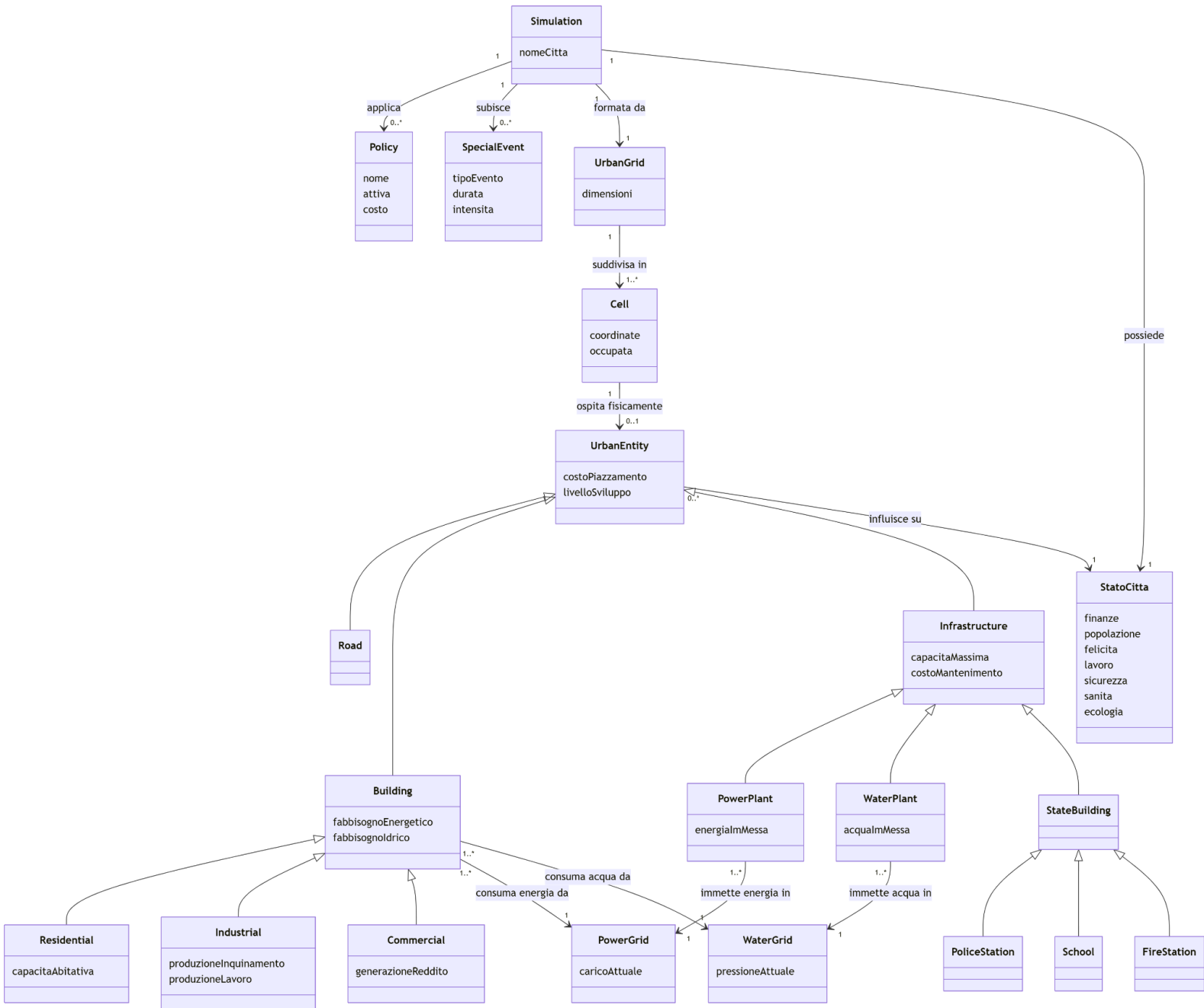


Documento di Design

1. Modello di Dominio (Domain Model)

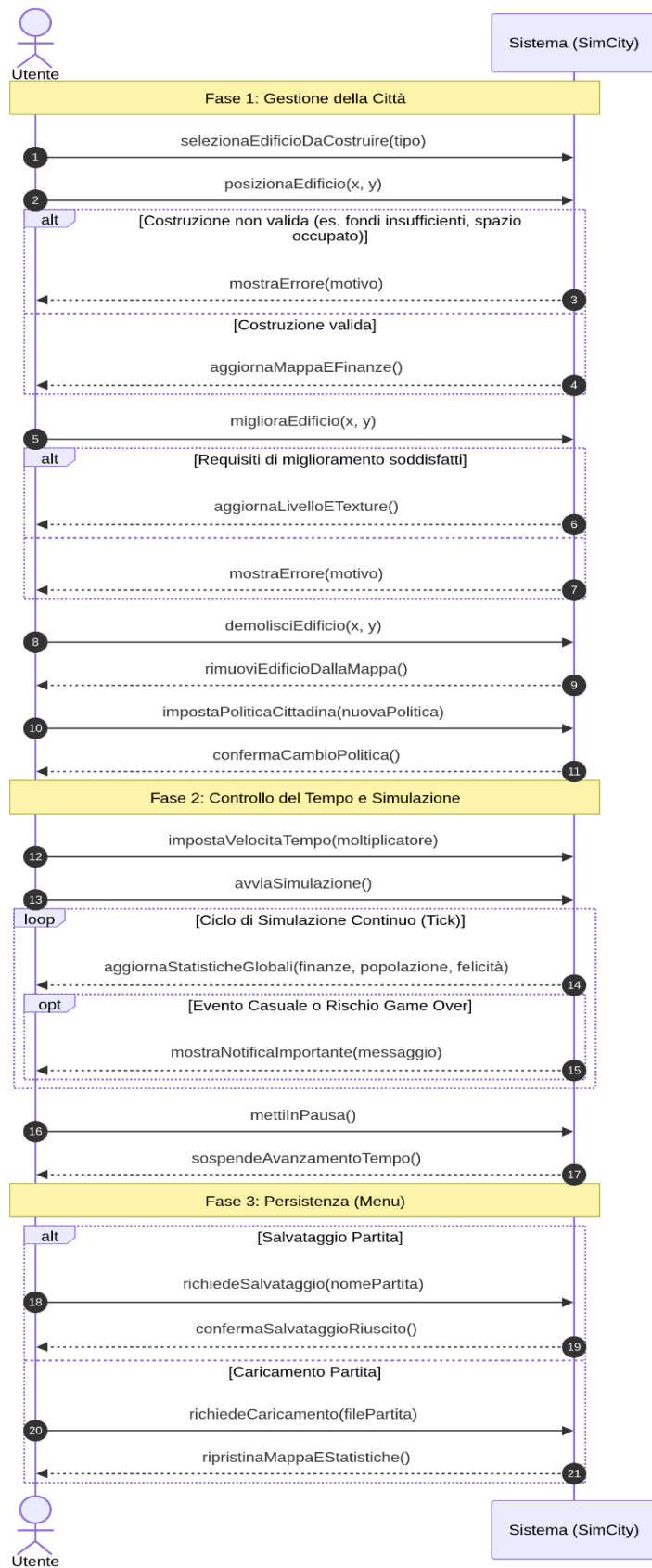


1. Glossario del Dominio

- **Città:** Rappresenta l'istanza globale della simulazione e contiene lo stato, la mappa territoriale, le politiche e subisce gli eventi.

- **StatoCittà:** Il contenitore delle 7 metriche centrali (Finanze, Popolazione, Felicità, Lavoro, Sicurezza, Sanità, Ecologia) che quantificano oggettivamente le prestazioni e le condizioni attuali della città.
- **PoliticaCittadina:** Una direttiva o regola attivabile (con un relativo costo) che modifica le regole di calcolo o i parametri della simulazione.
- **GrigliaUrbana:** La struttura di base della mappa, definita da una dimensione specifica.
- **Cella:** L'unità spaziale minima della GrigliaUrbana, localizzata tramite coordinate. Funge da "slot" che può ospitare al massimo una Entità Urbana, oppure rimanere libera.
- **EventoSpeciale:** Un'occorrenza dinamica che altera il normale flusso della simulazione per una certa durata e con una determinata intensità.
- **EntitàUrbana:** Astrazione base per qualsiasi elemento posizionabile su una Cella. Definisce le proprietà comuni: il costo di piazzamento e il livello di sviluppo.
- **Edificio:** Categoria di Entità Urbana che richiede connessioni alle reti di supporto (ha un fabbisogno energetico e un fabbisogno idrico) per funzionare. Si specializza in Zone:
 - **Residenziale:** Edificio dedicato all'incremento della popolazione tramite la sua capacità abitativa.
 - **Industriale:** Edificio che genera un impatto ecologico negativo (produzione inquinamento base) e un lavoro positivo.
 - **Commerciale:** Edificio focalizzato sulla generazione di un reddito base.
- **Infrastruttura:** Categoria di Entità Urbana destinata a fornire servizi o supporto logistico. Possiede un limite operativo (capacità massima) e richiede un costo di mantenimento. Si specializza in:
 - **EdificioElettrico:** Infrastruttura che immette energia nella Rete Elettrica.
 - **EdificioIdrico:** Infrastruttura che immette acqua nella Rete Idrica.
 - **Statale:** Edifici di servizio pubblico (Polizia, Scuola, Pompieri) dotati di specifiche capacità operative (agenti, studenti) che influiscono direttamente sulle metriche dello StatoCittà.
- **Strada:** Specifica Entità Urbana utilizzata per la viabilità spaziale.
- **ReteElettrica / ReteIdrica:** Entità logiche di distribuzione che tengono traccia rispettivamente del carico di energia e della pressione dell'acqua, bilanciando quanto immesso dai generatori e quanto consumato dagli Edifici.

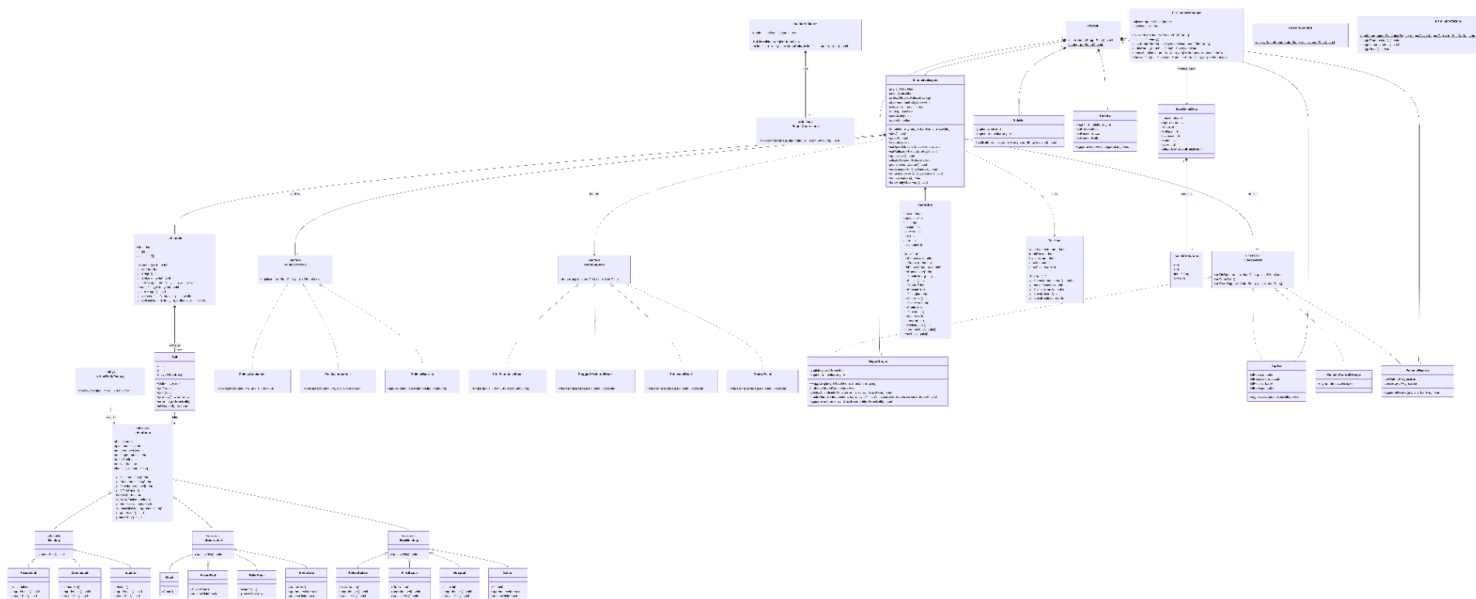
2. System Sequence Diagrams (SSD)



Il sistema è un motore di simulazione urbana basato su architettura a eventi, strutturato per gestire l'evoluzione dinamica di una città tramite un ciclo di simulazione discretizzato (*tick-based*). L'architettura adotta un approccio a componenti disaccoppiati: la logica di dominio (entità, griglia, stato) è separata dalla gestione dell'interfaccia utente (pattern Observer) e dalla persistenza dei dati (*PersistenceManager*).

Il motore coordina le interazioni tra gli oggetti della mappa, lo stato globale della città e le strategie di governo (pattern Strategy), garantendo che ogni entità elabori la propria logica interna in modo indipendente. La validazione delle azioni dell'utente, come il posizionamento degli edifici, è delegata a un sistema modulare di regole (*BuilderValidator*), garantendo la consistenza logica della griglia urbana. Il sistema è progettato per essere facilmente estensibile, consentendo l'integrazione di nuovi eventi, politiche o tipologie di edifici senza alterare il nucleo centrale del motore, vecchio.

3. Design Class Model



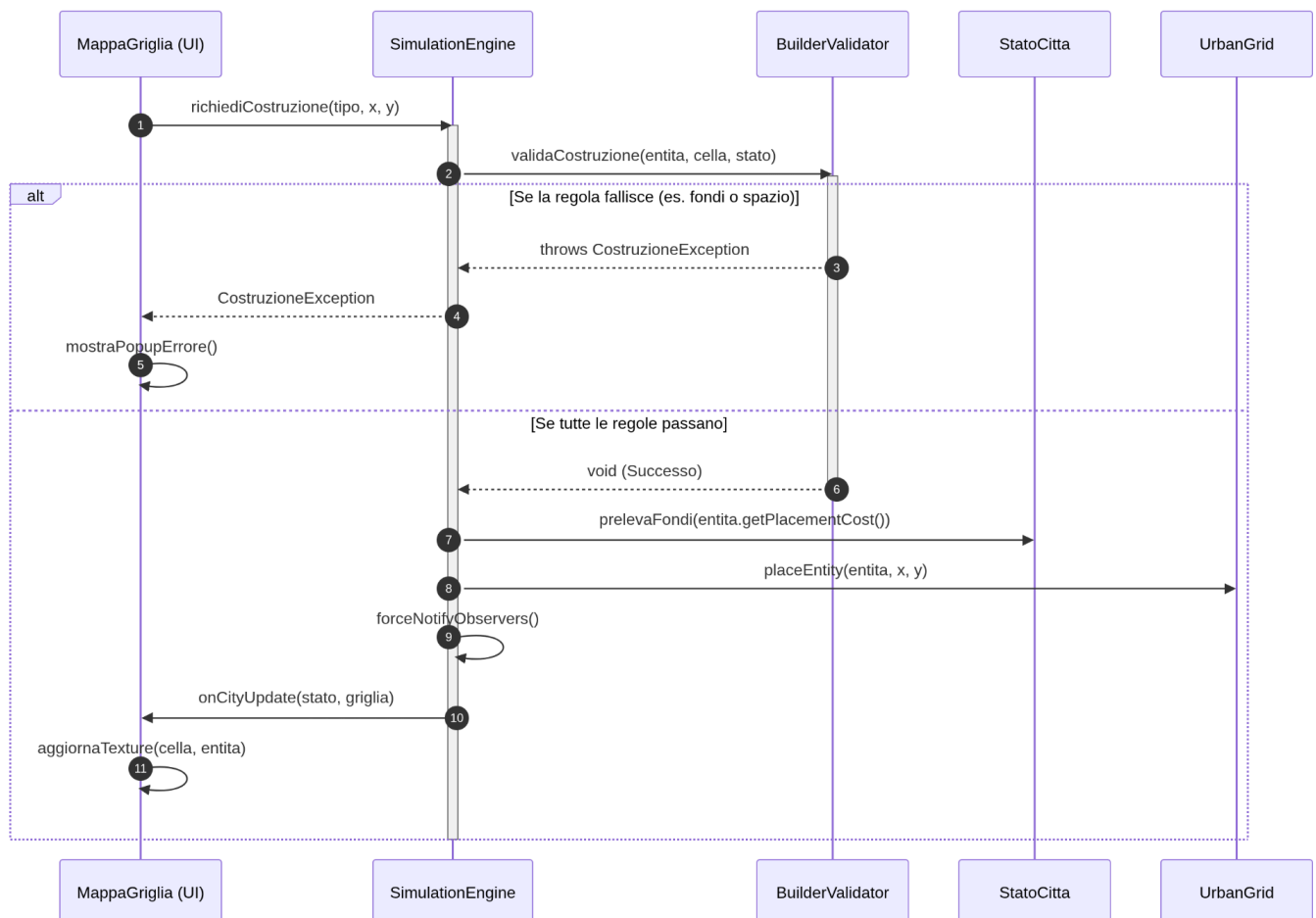
Le Scelte di Design Applicate:

- Separazione per Livelli (MVC): Il codice è diviso in Domain (Dati e Regole), Core (Motore), UI (Grafica) e Infrastructure (Salvataggi). Ogni pacchetto è isolato per mantenere il codice ordinato.
- Factory Pattern (UrbanEntityFactory): Per creare un nuovo edificio sulla mappa usiamo una Factory. Il motore non usa mai la parola chiave new direttamente, permettendo di istanziare edifici dinamicamente solo tramite il loro "nome".
- Observer Pattern: L'interfaccia utente (UI) si aggiorna automaticamente ad ogni "tick" di gioco mettendosi in ascolto del SimulationEngine tramite l'interfaccia CityObserver.

- **Strategy Pattern (Politiche):** Le diverse politiche cittadine (Industriale, Ambientale, ecc.) sono implementazioni di un'interfaccia `PoliticaStrategy`. Cambiare politica significa semplicemente scambiare l'oggetto strategia in esecuzione, senza riempire il motore di complessi cicli if/else.
- **Single Responsibility (Persistenza):** Il `SimulationEngine` fa solo il gioco. Il compito noioso di leggere e scrivere file JSON è delegato interamente a un `PersistenceManager` dedicato, che lavora usando oggetti leggeri di trasferimento dati (DTO).
- **Separazione per Livelli (MVC):** Il codice è diviso in Domain (Dati e Regole), Core (Motore), UI (Grafica) e Infrastructure (Salvataggi). Ogni pacchetto è isolato per mantenere il codice ordinato.

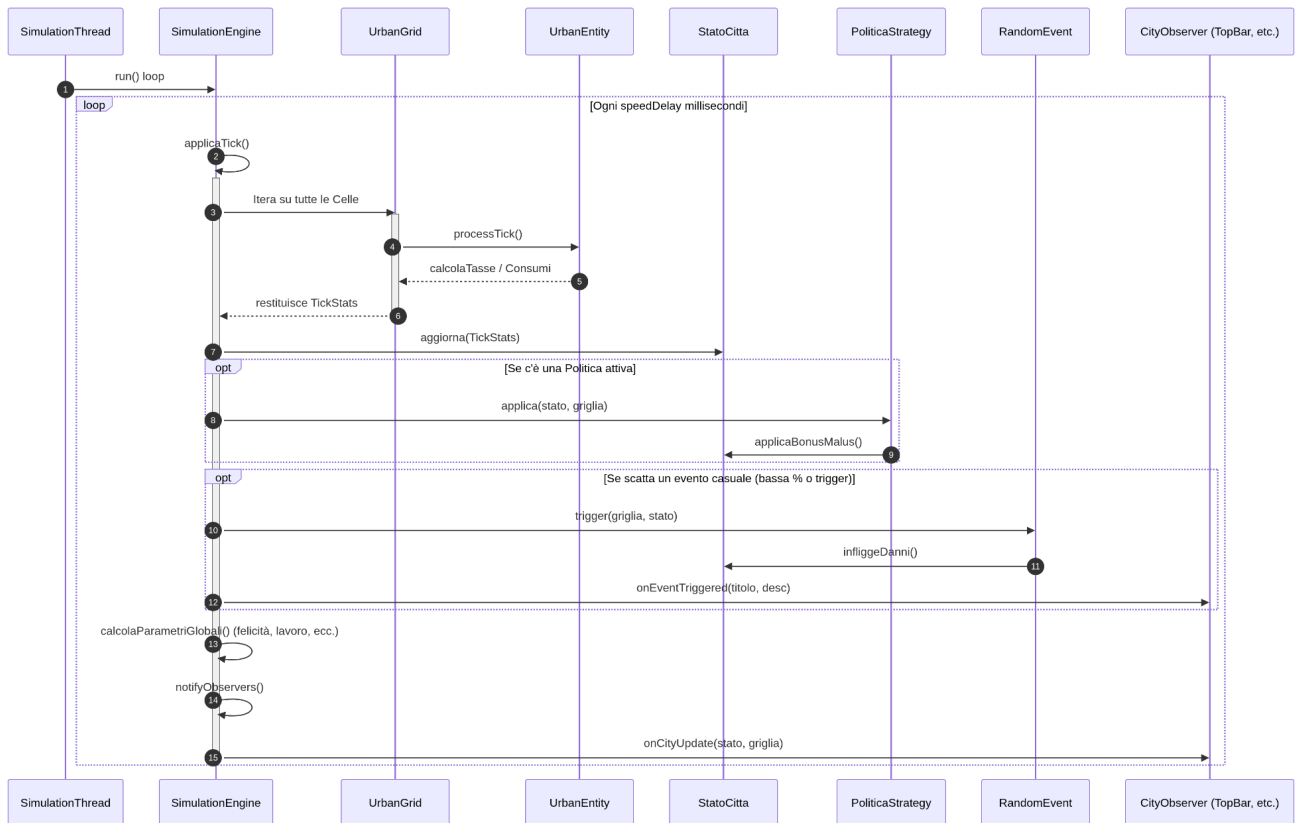
4. Internal Sequence Diagrams (ISD)

Costruzione edificio



Questo diagramma mostra come viene garantita l'integrità dei dati quando l'utente tenta un'azione. Invece di far controllare i requisiti (spazio, soldi) all'interfaccia o al motore, l'Engine delega la verifica al `BuilderValidator`. Solo se il validatore dà il via libera (senza lanciare eccezioni), il motore procede a prelevare i fondi dallo `StatoCitta` e a istanziare l'edificio nella `UrbanGrid`. Infine, forza un aggiornamento per ridisegnare la UI.

Avanzamento tick

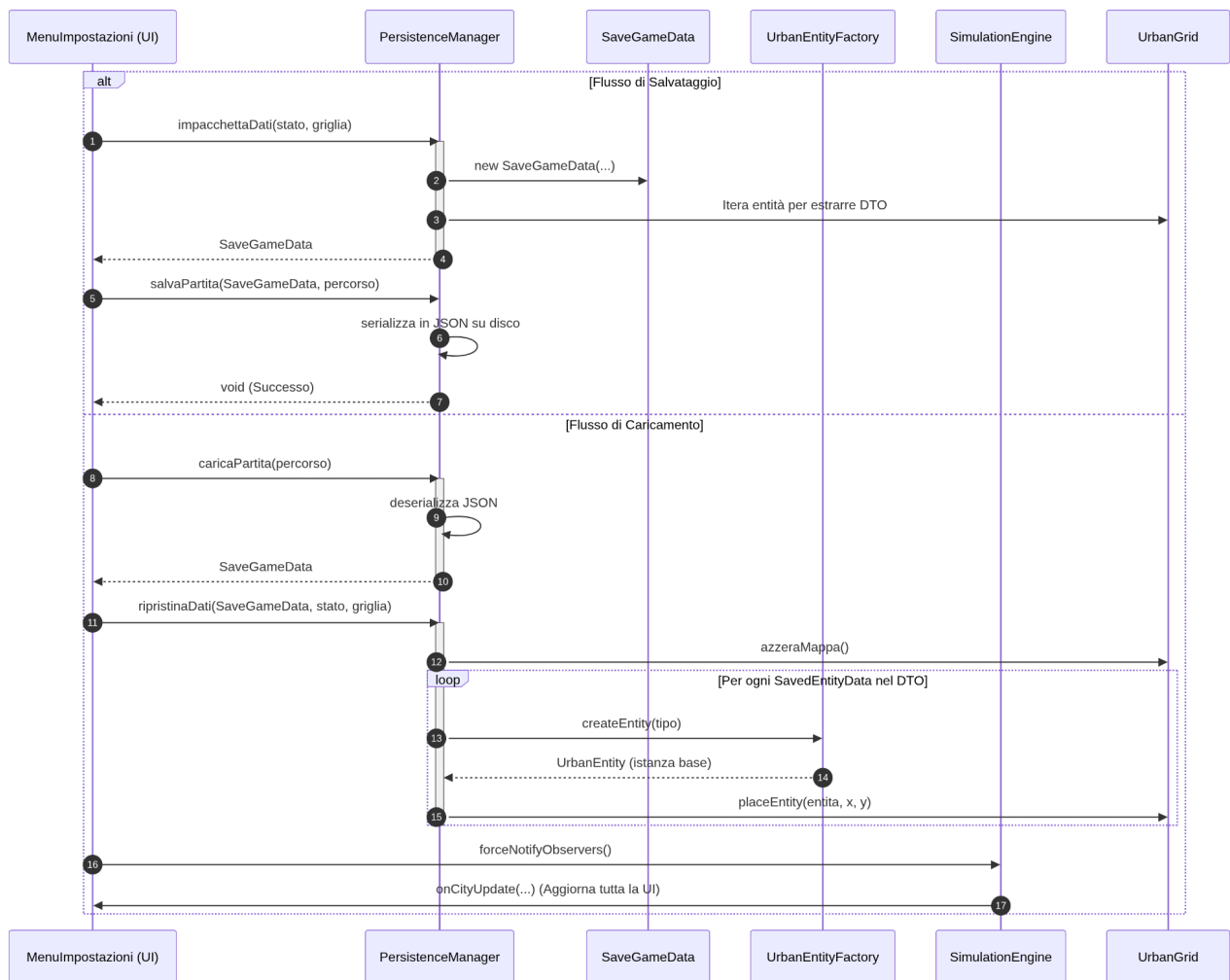


A intervalli regolari, l'Engine esegue un `applicaTick()`. Il diagramma evidenzia tre design pattern:

Delegation: La mappa itera sugli edifici chiamando `processTick()`, calcolando tasse e manutenzione.

Strategy: L'Engine applica la **PoliticaStrategy** attualmente attiva allo stato della città, modificandone i parametri senza usare blocchi if-else hardcoded.

Observer: Alla fine del ciclo, il motore chiama `notifyObservers()`. Le interfacce UI (che implementano **CityObserver**) ricevono i nuovi dati e si aggiornano da sole in modo disaccoppiato.



Questo diagramma descrive il disaccoppiamento tra il gioco in esecuzione e il file system. Nel salvataggio, l'Engine non c'entra nulla: la UI chiama il PersistenceManager, il quale impacchetta le coordinate e le statistiche in un DTO (Data Transfer Object) chiamato SaveGameData e lo serializza in JSON. Nel caricamento, per ricreare la mappa partendo da un file di testo, il PersistenceManager legge i DTO e utilizza la UrbanEntityFactory per istanziare dinamicamente gli oggetti corretti (Residenze, Ospedali, ecc.) da reinserire nella Griglia, prima di imporre un refresh alla UI.